

Valve Inventory

Technical Manual

Architecture, schema, and internals - for anyone extending the tool, scripting against the database directly, or just wanting to know how a feature actually works under the hood.

1. Architecture

One SQLite database, two independent front ends that read and write it directly - there is no server, no API layer, and no ORM. Both front ends import `valvelib.py` for the schema and shared logic.

File	Role
<code>valvelib.py</code>	Schema (SCHEMA string, executed via <code>executescript</code>), type-name normalisation (<code>norm()</code>), and the naming-convention classifier (<code>classify()</code>).
<code>valves.py</code>	Command-line front end. One <code>cmd_*</code> function per subcommand, dispatched via <code>argparse</code> .
<code>valves_gui.py</code>	Tkinter desktop front end. A single <code>App(ttk.Frame)</code> class holding four tabs' worth of widgets and handlers.
<code>build_db.py</code>	One-off converter from the original 38-tab spreadsheet. Already run; kept for provenance, not part of normal operation.
<code>snapshot.py</code>	Writes/restores the data/ text snapshot that stands in for the binary database in version control.
<code>fetch_datasheets.py</code>	Two-stage, rate-limited crawler/downloader for the local datasheet archive (<code>frank.pocnet.net</code> only).
<code>import_researched.py</code>	Parses a Claude research reply (block format below) and applies it to <code>valve_type</code> .
<code>test_smoke.py</code>	No framework - a flat script of <code>check()/check_true()</code> calls, exits non-zero on first failure.

2. Database schema

Six tables plus one convenience view. The tables are declared in `valvelib.SCHEMA` and created with `CREATE TABLE IF NOT EXISTS`; the view is declared separately in `V_STOCK_SQL` and rebuilt by `migrate()`. Both front ends call `V.init_db()` on every startup, which runs `SCHEMA` and then `migrate()`, so an existing database is brought up to the current schema in place with no separate migration step to run by hand - see "Migrations" below.

valve_type - one row per type, the reference library

Primary key `type_key` (normalised: uppercase, alphanumeric only - see `norm()` below). Columns: `name`, `function`, `family`, `base`, `pins`, `heater_v`, `heater_a`, `va_max`, `pa_max`, `gm`, `mu`, `power_out`, `freq_max`,

typical_use, equivalents (space-separated), datasheet_path, datasheet_url, confidence (inferred | confirmed), notes.

stock - one row per physical lot

type_key (FK to valve_type, ON UPDATE CASCADE), box, position, qty, manufacturer, condition, type1, type2, origin, test_values, other, date_added, notes. The box identifier is free text, not an enforced foreign-key relationship - the separate box table below just carries optional per-box location/label notes keyed on that same identifier.

position, type1, type2, origin, test_values and other were added in v1.4 and are nullable throughout: nothing reads them except to display, search and export them, so a database that never fills one in behaves exactly as it did before they existed. position is a single free-text grid reference ("B-12") rather than separate row and column columns, so it fits whatever scheme a box already uses; lot listings order by position IS NULL, position so un-positioned lots sort last instead of first.

Note: type1/type2 belong to the lot, not the type: they record what this particular glass is marked with, which is not necessarily the designation it's filed under. That is a different claim from valve_type.equivalents, which asserts something about the types themselves - so the two are deliberately not merged, and nothing writes one from the other.

socket - one row per lot of bases/sockets

base, box, qty, condition, notes. Split out from sundry deliberately, so a base type is a first-class, searchable thing rather than free text.

sundry / box

sundry is the general catch-all for non-valve, non-socket items (crystals, screening cans, chimneys). box holds per-box location/label notes, keyed on the same free-text box identifier used in stock and socket.

document - extra datasheets, links, and the general reference library

id, type_key (FK to valve_type, nullable), title, abstract, path, url, added. Deliberately additive rather than a replacement for valve_type.datasheet_path/datasheet_url, which remain the one "primary" sheet that every existing Open-datasheet code path already knows how to open - this table only ever adds to that, it's never read by the primary-sheet lookup. Whether a row is per-type or general-library is entirely down to type_key: NOT NULL puts it in that type's "Manage..." document list (DatasheetManagerDialog), NULL puts it in the Docs tab instead. path and url are both optional and independent - a link-only row (path NULL) is exactly how "note a project that mentions this valve, without downloading anything" gets recorded.

v_stock (view)

stock LEFT JOIN valve_type, exposing the commonly-needed combined columns (type name, position, type1/type2, origin, test_values, other, function, heater_v, pa_max, freq_max, base, datasheet_path) without repeating the join in every query.

It's the one object that can't be declared with an IF NOT EXISTS and left alone: it names stock's columns explicitly, so it goes stale the moment stock gains one, and `CREATE VIEW IF NOT EXISTS` would leave an older database on an older definition for ever. Hence it lives in `V_STOCK_SQL` rather than `SCHEMA`, and `migrate()` drops and recreates it every time.

Migrations

`migrate(con)` handles the one kind of schema change `CREATE TABLE IF NOT EXISTS` can't: a column added to a table that already exists. `ADDED_COLUMNS` maps a table name to the (column, declaration) pairs added after its first release; `migrate()` compares that against `PRAGMA table_info` and issues an `ALTER TABLE . . . ADD COLUMN` for whatever's missing, then rebuilds `v_stock`. It returns the list of columns it actually added, so an empty list means the database was already current.

Both steps are idempotent and safe to run on a current database, which is why `init_db()` can call it unconditionally on every startup. SQLite's `ADD COLUMN` only appends a nullable column to the table definition - existing rows are not rewritten and read back `NULL` for the new column - so the operation is cheap regardless of how many rows the table holds, and there is no window in which existing data is at risk. To add a column in a future version, append it to both `SCHEMA` (for fresh databases) and `ADDED_COLUMNS` (for existing ones).

Note: The `type_key` -> `stock.type_key` foreign key is declared `ON UPDATE CASCADE`. That matters in practice: renaming a `type_key` (e.g. correcting a mis-entered designation) is a single `UPDATE` on `valve_type`, done through a connection with `foreign_keys=ON` (`V.connect()` sets this) - `stock` rows follow automatically, no manual `UPDATE stock` needed.

3. Type-name normalisation and classification

`norm(name)`

Uppercases, strips a leading `JAN/JAN-/CV-` service prefix, then strips everything that isn't `A-Z0-9`. "jan 7289" → "7289"; "PY 4-400" → "PY4400". This is the `type_key` used everywhere - display names keep their original punctuation in name.

`classify(name)`

Infers function/heater/family from the designation alone, trying each national naming convention in turn until one matches: a curated `KNOWN` table (hand-entered for designations no rule covers) first, then GEC beam tetrodes (KT-prefix), European Mullard/Philips codes, British Mazda/Brimar codes, Russian codes, American RETMA codes, British GEC/Osram codes, then transmitting-type patterns (4CX/3-500Z-style). Returns whatever it's confident about and leaves the rest blank - see the code comments for exactly which patterns are accepted and why (several national schemes collide on the same letter/number patterns, so order and specific guard conditions matter).

Note: `classify()` output is always `confidence='inferred'`. Nothing sets confidence to confirmed except a human (Save + confirm in the GUI, `--confirm` on the CLI) or `import_researched.py` applying a research result that wasn't hedged.

4. GUI internals worth knowing

Equivalents-aware search

`run_search()` builds the normal filtered query, then `add_equivalent_rows()` checks whether the Text field names an exact type_key; if so it pulls in stock for every type_key that either appears in that type's own equivalents field, or has this type's key in *its* equivalents field (checked both directions in Python, not SQL, since equivalents is unstructured free text). Matched rows get a match label and the equiv tree tag.

Similar-types suggestions

`find_similar()` buckets the current type's function text via `function_group()` (a simple first-match keyword scan against `valvelib.FUNCTION_GROUPS`), then scans every other held type in the same bucket, requiring every field both types have a value for (`va_max`, `pa_max`, `gm`, `mu`, `power_out`, `freq_max`) to be within `SIMILAR_TOLERANCE` (50%) of the reference. Heater is deliberately excluded from the match test and checked separately as a flag - a different heater rating doesn't rule out a substitute, it just needs a note.

Browse tab - Category vs. Function

The raw function text is specific enough that almost every type has a unique value, which makes it useless as a browse facet. `browse_category()` maps it to a coarser, purpose-built bucket list (`BROWSE_CATEGORIES`) instead - compound types are checked before the simpler categories they'd otherwise match as a substring ("triode-pentode" before "triode"). `variable_mu` is a second derived, orthogonal flag (checks for "variable-mu" / "remote cutoff" wording), since that property cuts across pentode and tetrode rather than being its own category.

Because category/`variable_mu` aren't real columns, the whole Browse tab filters in Python over the full (small, ~250-row) type list rather than building SQL per field - see `pb_load_all()` / `pb_matches()` / `pb_refresh_dropdowns()`. At this scale that's faster to reason about than parallel SQL and Python category logic, and cascading dropdown options (`pb_matches(..., exclude=field)`) fall out of the same predicate function for free.

Repair Bench - composition over duplication

Nothing about identifying an unknown valve or proposing a substitute is new logic - the tab (`rb_*` methods) is built entirely from pieces the other tabs already needed: `find_similar()` for substitute candidates (reused as-is, then filtered to `held_qty > 0` - a substitute you don't have isn't useful mid-repair), `do_open_sheet(row)` / `do_lookup(site, row)` for datasheet and web lookups (both already took an optional explicit row dict rather than always reading the Valves tab's selection, precisely so other tabs could call them), and the same bidirectional equivalents scan used by the Valves tab's search (factored out here as `rb_find_matches()` rather than shared directly, since the Valves tab version is itself embedded in a larger SQL query builder). `save_type()` was split into `apply_type_fields(key, field_vars, notes_widget, confirm)` plus a thin wrapper specifically so Repair Bench's Save button could reuse the exact same validate-and-write logic against its own, separate form widgets.

5. The research/import pipeline

Both the electrical-parameters prompt and the datasheet-download prompt are generated from the live gap list at the moment you ask for them (Tools menu), not hard-coded - re-running them later reflects whatever's still unconfirmed then.

Expected reply format

```
TYPE__NAME
function:
base:
pins:
heater_v:
heater_a:
va_max:
pa_max:
gm:
mu:
power_out:
freq_max:
typical_use:
equivalents:
source_url:
confidence_note:
---
```

import_researched.py's `parse_blocks()` splits on the `---` separator, normalises the first line of each block through the same `norm()` used everywhere else, and pulls a bare number out of each numeric field with a permissive regex (so "250 V" or "~250" still parses). A block whose only non-blank field is equivalents is skipped entirely, on the basis that "we already knew that" isn't a research result worth recording.

Confirmed vs. lead-only

`apply_records()` checks `confidence_note` against a fixed list of hedge phrases (`HEDGE_WORDS`: "could not", "low confidence", "plausible", "unconfirmed", and similar). If any match, the fields are still written - the data is a genuine lead worth keeping - but confidence stays `inferred` rather than flipping to `confirmed`. This is the same policy applied by hand throughout this collection's own research pass; see the notes field on any type for a worked example.

6. CLI command reference

Command	Purpose
box BOX	List a box's contents.
find TYPE	Which boxes hold a type (follows equivalents).
show TYPE	Full reference record for a type.

Command	Purpose
search --function .. --heater .. --pa ..	Parametric search; numeric flags take a bare value or a >, <, >=, <= comparison. --position, --origin and --alt (Type 1 / Type 2) filter on the lot's own fields; --text covers those too.
add TYPE --box N [--qty --maker --condition --notes --position --type1 --type2 --origin --test --other]	Add stock; creates the type automatically if new, and reports the lot id it created.
edit LOT_ID [same options as add]	Change one lot in place; only the options given are written, and " clears a field. Lot ids come from the ID column of box/find/show.
import-csv FILE	Bulk-add stock from a CSV (see upload_template.csv). Only type and box are required; any other column may be blank or absent.
take TYPE [--qty --box]	Remove stock you've used.
move TYPE --frm A --to B [--position]	Move a type between boxes; the position is replaced or cleared, since it belonged to the old box.
bases [--base --box]	List valve base/socket stock.
sock-add / sock-take / sock-move	Same idea as add/take/move, for bases/sockets.
docs [--type TYPE]	List reference documents - the general library by default, or one type's primary datasheet plus its extra documents/links with --type.
set TYPE --pa .. --va .. --confirm	Edit a type's reference parameters; --confirm marks it confirmed.
merge SRC DST [--yes]	Fold one type into another - moves stock, records SRC as an equivalent of DST. Dry-run without --yes.
dupes	Candidate duplicate type pairs, for review before merge.
scan [--archive]	Link local datasheet files into the database by filename.
sheet TYPE [--open]	Locate (and optionally open) a type's local datasheet.
gaps [--limit]	What still needs data, ordered by quantity held.
stats	Collection summary.
export [path]	Write an .xlsx snapshot (requires openpyxl).

7. Version control and the snapshot design

`valves.db` is gitignored. SQLite databases are binary blobs - git can store them but every single-row edit rewrites the whole file, so diffs are meaningless and history is bloated. `snapshot.py` writes the same content as plain CSV (one file per table) plus a full SQL dump (`valves.sql`, via `con.iterdump()`), all under `data/`. That's what's actually committed - readable diffs for every change to the collection, and a restore path that doesn't depend on the SQLite file format being stable across versions.

Note: iterdump() emits tables in alphabetical order, which lists stock rows before valve_type exists. restore() explicitly runs with foreign_keys=OFF while loading the dump for exactly this reason, then reports (rather than silently ignoring) any row that fails PRAGMA foreign_key_check afterward.

8. Privacy note on distributing this data

The typical_use and notes fields carry descriptive text originally gathered from r-type.org during the original conversion - not something to republish freely. snapshot.py --strip-notes (and the equivalent checkbox in File > Export archive and tools) omits those two fields from the export while leaving every classification, parameter, and box location intact. Datasheet PDFs are gitignored and never included in an export for the same reason - they're third-party files, meant to be rebuilt locally by whoever needs them.

9. Extending the tool

- **New reference field** - add the column to SCHEMA *and* to ADDED_COLUMNS in valvelib.py (SCHEMA builds fresh databases, ADDED_COLUMNS brings existing ones up to it - see "Migrations" above), then to TYPE_FIELDS in valves_gui.py (drives the detail-panel form) and ALL_FIELDS in import_researched.py if research prompts should be able to fill it. Nothing else is needed: migrate() runs on every startup and applies the ALTER TABLE itself.
- **New per-lot field** - as above for the schema, then LOT_FIELDS in valves.py (the add/edit options and the CSV importer) and LOT_FIELDS in valves_gui.py (the Add stock and Edit lot forms). Add it to STOCK_SELECT/STOCK_COLS in valves_gui.py to show it as a results column, to STOCK_COLS in snapshot.py so it's committed, and to cmd_export in valves.py so it reaches the spreadsheet.
- **New CLI command** - one cmd_*(con, a) function in valves.py plus an add_parser() block; follow an existing command for the pattern.
- **New GUI tab** - add a _build_x_tab(root) method, call it from App.__init__ alongside the existing four, and give it its own prefixed method names (px_*) to avoid colliding with the other tabs' state.